BROWSER TRUST / REQUEST INTENT / SECURE DESIGN

CSRF Explained

A visual guide from browser basics to finding, proving, and preventing request forgery

---

A COMPLETE VISUAL LEARNING KIT

Understand the browser before the vulnerability.
Find the failure. Prove the effect. Repair the boundary.
Practice with fictional accounts on your own computer.

Haroon Shahid

First edition / Version 1.0 / 24 October 2026

Before you begin

CSRF Explained

A visual guide from browser basics to finding, proving, and preventing request forgery

By Haroon Shahid. First edition, version 1.0. 24 October 2026.

Copyright 2026 Haroon Shahid. The purchased publication is licensed for personal use under the included LICENSE.txt. The separately identified lab code is MIT licensed. The free preview may be shared unchanged with attribution. No resale or public redistribution of the full paid publication is granted.

This edition teaches responsible browser-security analysis through fictional, local examples. It does not certify professional competence. Framework and protocol chapters are version-sensitive review patterns. Read the Validation Report for executed test scope.

PDF code may wrap for page fit. The offline HTML and source files retain original code lines. The HTML reading edition also provides responsive text and native answer controls; no full PDF accessibility conformance is claimed.

Free preview: introduction, roadmap, and Chapters 1-3. The full package also includes all 48 chapters, answered reviews, offline HTML, practice workbooks, the local lab, and reference sheets.

Contents

Before you begin 2

Contents 3

Introduction 4

Learning roadmap 5

01. The web is a conversation 6

02. HTTP: what a request actually says 8

03. Three maps: origin, site, and cookie scope 10

Sources and further reading 13

Introduction

Amina has signed in to a notes application. She opens a different page. A moment later, her display name changes. No one stole her password. The other page did not need to read her private account. Her browser carried authority, and the application accepted an instruction without the right boundary.

That is the puzzle this book will help you solve. We begin before tokens: messages, URLs, cookies, sessions, origins, sites, and the difference between sending a request and reading its answer. Then we build complete local proofs, repair the failed contracts, and review harder systems where apparently valid proof can still accompany the wrong operation.

You need no prior security knowledge. Read Chapters 1-12 in order if these ideas are new. The short code examples are explained as they appear. You can read the book without running code; practical skill grows when you also complete the predictions and local missions. The advanced chapters expect the foundations and point back to them.

Use a four-column notebook: identity, permission, request proof, and effect. Before each experiment, predict what the browser and server will do. Afterward, record what happened and which part of your model changed. This is how a collection of facts becomes useful judgment.

All executable demonstrations use fictional accounts and fixed loopback destinations. Work only in environments you own or are authorized to assess. The book includes attack mechanics because a defense is easier to understand when its failure is observable. The goal is a reproducible, bounded proof followed by a correct repair and a working legitimate path.

This edition is a substantial learning resource, not a certification, a promise of earnings, or a claim that one book can exhaust every future browser behavior. Framework and protocol guidance is version-sensitive. The Validation Report distinguishes executed lab tests from documented review patterns. The source catalog provides a path for checking and updating your knowledge.

Read slowly enough to predict. Practice carefully enough to explain. Return to the diagrams when the details become confusing.

Learning roadmap

Stage	Read and practice	Demonstrate before advancing
Foundations	Chapters 1-12; Predictions 1-10	Draw a request and cookie lifecycle; distinguish origin, site, and scope
Evidence	Chapters 13-20; Missions 1-10	Prove a harmless write with identity, source, credential, and before/after evidence
Defenses	Chapters 21-28; Missions 11-18	Explain token binding, exact sources, metadata fallback, and pre-effect enforcement
Difficult boundaries	Chapters 29-36; Missions 19-20	Trace a valid-proof deputy and a protocol or asynchronous operation on paper
Framework review	Chapters 37-42	Find the actual version, endpoint coverage, exclusions, and test settings
Independent work	Chapters 43-48; Review Clinic	Produce a reproducible capstone report and a narrow, usable repair plan

Choose your route

New learner: follow the order and explain each chapter's takeaway aloud. Developer: use Chapters 1-12 to check assumptions, then focus on the defense and framework parts while completing at least one browser proof. Reviewer: use the discovery workflow and evidence notebook, but return to foundations whenever a request result surprises you. Team mentor: assign predictions before demonstrations and compare explanations afterward.

Study for retention

Use a 30-45 minute session: read one chapter, sketch its map without looking, answer the three questions, then check the explanation in Appendix A. Repeat missed questions after a day and a week. Revisit one advanced case after a month. This is a suggested rhythm, not a claim that expertise arrives on a fixed schedule.

A practical stopping rule

Before moving on, be able to define the main idea, explain the analogy's limit, predict one example, name one common error, and connect the lesson to the next boundary. If you cannot, use the visual and a smaller experiment. Reading faster is not the same as learning faster.

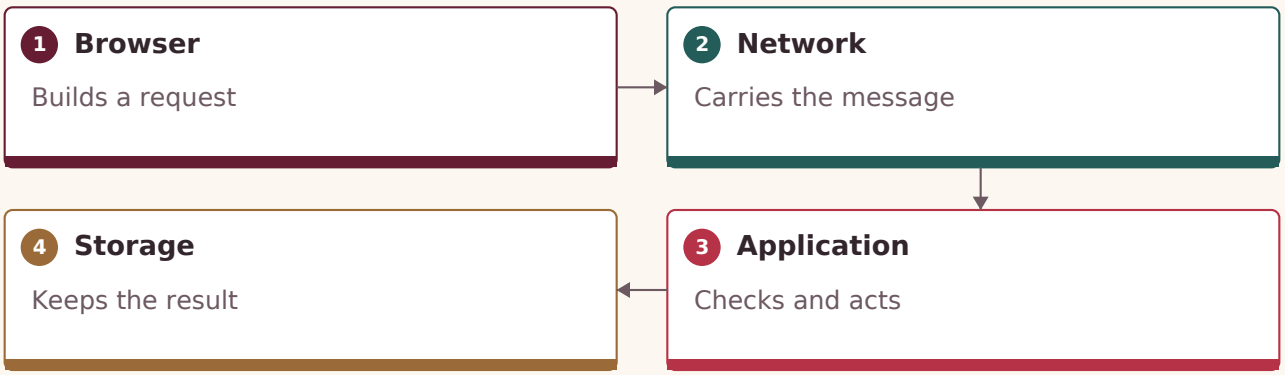
I. UNDERSTAND THE BROWSER

01. The web is a conversation

Learn to: Identify a client, a server, a URL, and a request; explain what HTTPS does.

Think of it this way: A browser is a customer at a service desk. A request is an order. The server can follow the order without knowing who wrote the instructions.

Figure 01. The working model



A web action crosses several components. An encrypted journey still needs a valid reason to act.

Begin with one click

A browser is a program that opens web pages. A server is a program that answers requests. The internet is the network that carries their messages. A web application adds behavior: it may save a note, change a password, or start a payment. The database is one possible place where it keeps information.

Type `https://notes.example/profile` into the address bar. The URL is the address of a resource. `https` is the scheme. `notes.example` is the host name. `/profile` is the path. A query begins with ?, as in `?tab=settings`. A fragment begins with #. The fragment normally stays in the browser rather than becoming part of the HTTP request target.

DNS is a directory that helps turn a host name into a network address. A connection carries messages to a service, usually at a numbered port. The default HTTPS port is 443. A local learning server can use a different port. Two different ports can be different web origins even on the same computer. We will make that distinction precise in Chapter 3.

Follow the message

The browser requests a page. The server answers with a status, headers, and usually a body. HTML describes the page structure. CSS controls its appearance. JavaScript can react to clicks and send more requests. A single visible page may involve many server messages. A tab is a viewing context; it is not a separate account or cookie jar in an ordinary browser profile.

An API is an interface that programs call. A page and an API often reach the same application rules. Moving a change from an HTML form to an API does not change the need to check who may cause it.

HTTPS protects traffic in transit and authenticates the server through the browser's certificate checks. It does not prove that the user wanted a particular request. A harmful request can travel

through a correctly encrypted connection. Keep transport security and application intent as separate questions.

Work through Cedar Notes

Cedar Notes is the fictional application used throughout this book. Amina is signed in. The page contains a form to change her display name. Amina clicks Save, the browser sends a message, and the application updates her record. Later, we will ask whether another page can cause a similar message. No real account or production service is needed to explore that question.

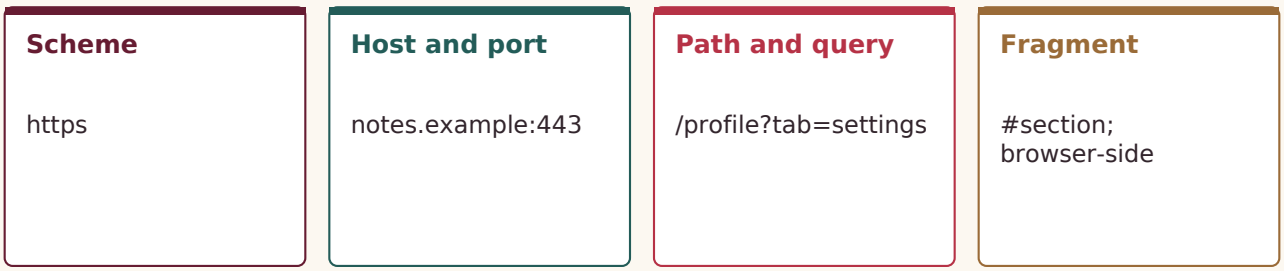
Avoid this mistake

Do not call every request a page load. Images, background API calls, navigations, and forms make requests in different ways. The mechanism affects credentials and browser checks.

Practice and reflection

Open Developer Tools in a page you own. Select Network, reload, and identify the main document and one extra resource. Record the host, method, and status. Do not share a raw capture containing cookies or tokens. Explain which part is the client and which part is the server before moving on.

Figure 01b. Read a URL as separate fields



Summary and key takeaway

A request is a message, not proof of a deliberate user decision.

Check your understanding

1.1 Does HTTPS prove that Amina clicked Save?

1.2 Does a fragment normally reach the server?

1.3 Do two ordinary tabs automatically isolate login cookies?

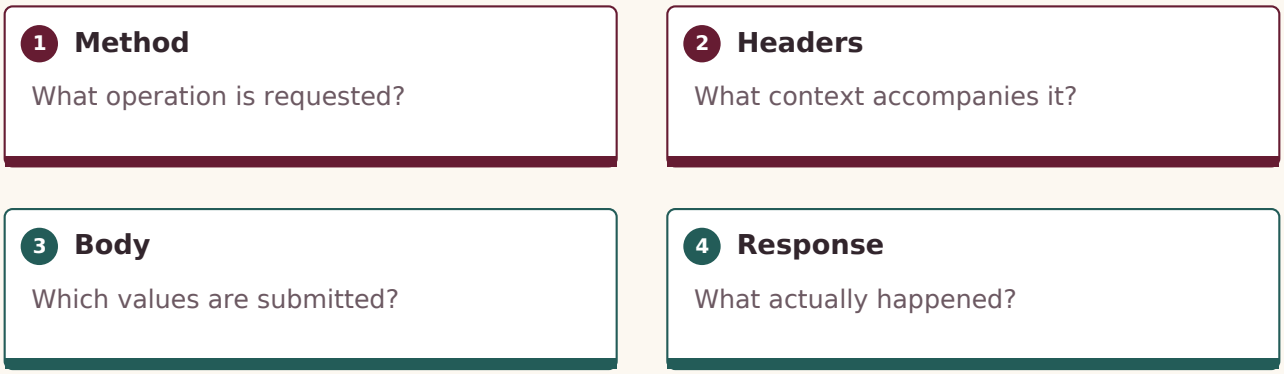
Answers appear in the full ebook. Sources: R01, R02

02. HTTP: what a request actually says

Learn to: Read methods, headers, bodies, and status codes; distinguish safe from idempotent.

Think of it this way: An envelope has a destination and labels. Its contents hold the order. Changing a label does not make an unsafe order safe.

Figure 02. The working model



Inspect the request and the resulting state. Neither a method name nor a status code proves security.

Read a small request

HTTP is the message protocol of the web. A method states the requested operation. The target identifies the resource. Headers carry metadata. A body can carry submitted data. HTTP/2 and HTTP/3 encode these parts differently from the readable HTTP/1.1 view below, but the application concepts remain useful.

```
POST /profile/name HTTP/1.1
Host: notes.example
Content-Type: application/x-www-form-urlencoded
Cookie: sid=demo-session

name=Amina
```

POST is the method. `Content-Type` describes the body format. The cookie is a credential candidate. The server must still validate the session, permission, request source or proof, and data. A header name is case-insensitive. A cookie name or token value follows its own rules; do not apply one comparison rule to every field.

Safe is a promise about behavior

A safe method requests essentially read-only semantics. GET should not ask the application to perform a user-visible mutation such as changing a recovery address. Servers may still log a read. Idempotent means repeating the same request has the same intended effect as sending it once. PUT and DELETE are defined as idempotent, but they can change important state. Safe and idempotent are different properties.

POST is not a security control. A cross-origin form can issue a POST. Conversely, an unsafe GET makes attacks easier because links, navigations, previews, and automated retrieval may trigger it.

The repair is to use appropriate method semantics and enforce intent on mutations.

Read the response carefully

`200` means a successful HTTP response, not necessarily that the requested business change happened. `302` and `303` often lead to a new navigation. `307` and `308` preserve the method and body across redirects. `401` indicates missing or invalid authentication; `403` indicates refusal; `405` rejects a method; `415` rejects a media type. An application can misuse these codes, so verify state as well as status.

An HTML form usually sends URL-encoded fields, multipart fields, or plain text. JSON is a structured text format often used by APIs. A parser turns received bytes into application values. If two middleware layers parse or interpret the same request differently, a security check can protect one meaning while the action uses another.

A concrete design decision

For Cedar Notes, GET `/profile` reads the current name. POST `/profile/name` changes it. A rejected change returns a clear error and leaves the name untouched. A successful change redirects to a read-only page. Replaying a request after a network timeout is a separate reliability problem; an idempotency key can address duplicates but is not a CSRF token.

Practice and reflection

Write two endpoint contracts: one read and one mutation. State the accepted methods, media types, authentication, and required proof. Then ask whether an automatic link preview could cause an effect.

Figure 02b. Separate four parts of request evidence



Summary and key takeaway

Safe describes requested semantics. Idempotent describes repetition. Neither replaces intent checks.

Check your understanding

2.1 Is DELETE safe because it is idempotent?

2.2 Does a 200 response prove a CSRF mutation?

2.3 Why avoid state changes on GET?

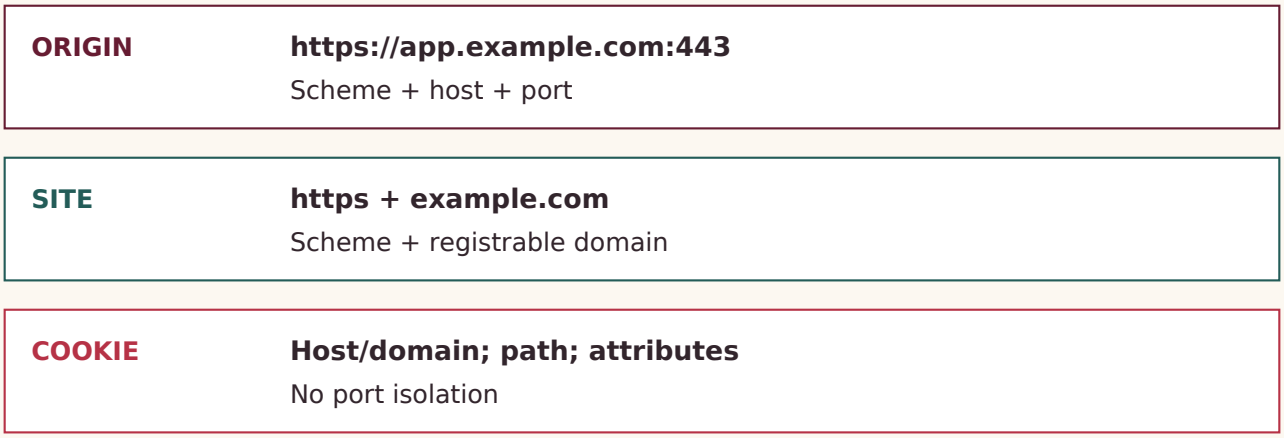
Answers appear in the full ebook. Sources: R03, R04

03. Three maps: origin, site, and cookie scope

Learn to: Compare origins, schemeful sites, and cookie matching without mixing them.

Think of it this way: A building, a neighborhood, and a delivery area are three maps of the same city. They answer different questions.

Figure 03. Three maps of one request



Use all three maps. A same-site request can cross an origin boundary.

Origin is a precise address boundary

For ordinary HTTPS URLs, an origin is the scheme, host, and effective port. `https://app.example.com` and `https://app.example.com:443` have the same origin. Changing to port 8443 changes the origin. Changing the host to `help.example.com` also changes it.

An opaque origin is an origin without a normal reusable tuple, as can happen in a sandboxed document. Its serialized Origin header can be `null`. The word `null` does not identify one trusted website. Many unrelated contexts can produce it.

Site groups related origins

A schemeful site normally combines the scheme and registrable domain. A registrable domain is the part that an organization can register under a public suffix. For `shop.example.co.uk`, the suffix is `co.uk` and the registrable domain is `example.co.uk`. The Public Suffix List matters; taking the last two labels is incorrect. IP-address hosts use the host itself rather than a made-up registrable domain.

`https://app.example.com` and `https://help.example.com` are cross-origin but same-site. The HTTP version of one is a different schemeful site. Ports distinguish origins but not sites. Site comparisons are used by SameSite cookies and Fetch Metadata; they do not grant general DOM or API read access.

Cookie scope is another rule set

A cookie without a Domain attribute is host-only. It matches that host. A permitted Domain attribute can include subdomains. Path narrows delivery by request path, but is not a strong security boundary between applications. Cookies are not isolated by port. Secure restricts delivery to secure transport, subject to browser handling of local development exceptions.

From and to	Origin relationship	Site relationship
https app.example.com to same host:8443	Different	Same
https app.example.com to https help.example.com	Different	Same
https app.example.com to http app.example.com	Different	Different schemeful site
https app.example.com to https outside.test	Different	Different

Worked example: delivery follows the target

Suppose a cookie was set by `https://app.example.com` without Domain and with Path=/. A page at `https://help.example.com` submits a form to `https://app.example.com/profile`. The form's source and target are different origins but normally the same site. The cookie's host-only match is evaluated against the target host, app.example.com. The source page does not have to receive or read that cookie to cause the browser to consider sending it to the target.

Now change the target to `https://help.example.com/profile`. The app's host-only cookie does not match that sibling host. Change the target instead to `https://app.example.com:8443/profile`. The origin changes, but the cookie is still not port-isolated. Whether it is actually sent also depends on its other attributes and the request context, which the next chapters teach.

This is why a diagram with only two circles labelled trusted and untrusted is often too vague. Write the source origin, target origin, target cookie scope, and site relationship separately. First decide which browser rule uses which map. Then observe the request. A cookie missing from one experiment can reflect a scope rule rather than an application intent check.

Use the maps in the local lab

The lab app uses `https://127.0.0.1:8767`. Its peer is available at `https://localhost:8768` for a cross-site case and `https://127.0.0.1:8768` for a same-site, cross-origin case. Cookies for 127.0.0.1 are not port-isolated. The peer deliberately discards incoming cookies. A truly hostile service on the same host could receive those cookies; this lab arrangement does not model cookie secrecy between sibling host names.

Practice and reflection

Draw these three boundaries for your own training application. For each trusted subdomain, record who can deploy code there. A same-site label is a relationship, not a statement that the other application is safe.

Figure 03b. Which boundary does the control inspect?



Summary and key takeaway

Cross-origin does not necessarily mean cross-site, and cookies are not port-scoped.

Check your understanding

3.1 Are app.example.com and help.example.com always different sites?

3.2 Does changing a port isolate cookies?

3.3 Can Origin: null be safely allowlisted as one partner?

Answers appear in the full ebook. Sources: R05, R06, R07

Sources and further reading

Research cutoff: 24 October 2026. Source aliases are grouped. The catalog includes direct sources and targeted further reading. Current documentation can change; consult release-specific material and errata. Historical sources are not a current browser compatibility table.

R01 - MDN: How the web works

Foundation reading: clients, servers, and network messages.

https://developer.mozilla.org/en-US/docs/Learn_web_development/Getting_started/Web_standards/How_the_web_works

R02 - WHATWG URL Standard

Living standard; URL parsing and origin concepts.

https://url.spec.whatwg.org/

R03 - RFC 9110: HTTP Semantics

Published standard, June 2022. Methods, status, redirects, safe and idempotent semantics.

https://www.rfc-editor.org/rfc/rfc9110.html

R04 - RFC 9112: HTTP/1.1

Readable message syntax; newer HTTP versions use different framing.

https://www.rfc-editor.org/rfc/rfc9112.html

R05 - web.dev: Same-site and same-origin

Browser-vendor explanation of two different boundaries.

https://web.dev/articles/same-site-same-origin

R06 - Public Suffix List

Why registrable domains cannot be found by taking the last two labels.

https://publicsuffix.org/learn/

R07 / R12 / R63 - RFC 6265: HTTP State Management Mechanism

Published 2011 cookie specification. Do not attribute later SameSite rules to this original RFC.

https://www.rfc-editor.org/rfc/rfc6265.html